

Algorithms for Data Science

Lecture #5: Divide & Conquer

Mateusz Buczyński

University of Warsaw

18.03.2026

Thanks to dr Krzysztof Fleszar for the base material.

Divide and Conquer

Core idea

Divide a big problem into smaller subproblems.

Typical workflow

- Split the original task into manageable parts.
- Solve each part (often recursively).
- Combine partial solutions into the final answer.

Decision rule

If a problem is too hard to solve directly,
divide it first.

Divide and Conquer: Generic Algorithm

Idea

For a problem of size n , choose between a **base case** and a **recursive case**.

If n is very small

- Solve directly.
- Use a simple method (e.g., brute force).

Otherwise

- Divide into a subproblems.
- Each subproblem has size at most $\frac{n}{b}$.
- Solve subproblems recursively.
- Combine their solutions.

Divide and Conquer: Structure of Work

Recursive pattern

Problem size $n \longrightarrow a$ subproblems of size $\leq \frac{n}{b} \longrightarrow$ combine

Interpretation of parameters

- a : how many smaller problems we create,
- b : how much smaller each one is.

Decision rule

Keep applying the recursive step until the problem becomes small enough to solve directly.

Recall any examples?

Warm-up question

Can you recall examples of divide-and-conquer from:

From the lecture

- Which examples did we discuss in class?
- What was the *divide* step?
- What was the *combine* step?

From exercises

- Which problems used the same pattern?
- What subproblems were solved recursively?
- How was the final answer assembled?

Small Example: Binary Search

Current search interval

We focus on a sorted array segment indexed from l to r .

$$n = r - l + 1$$

Choose the middle index m .

Left half

Indices from l to $m - 1$

Size at most $\frac{n}{2}$

Right half

Indices from $m + 1$ to r

Size at most $\frac{n}{2}$

Key point

At each step, binary search keeps only one of these halves.

Binary Search as Divide-and-Conquer

Recursive decision

If $l < r$ (problem not small), reduce to one subproblem of size $\leq \frac{n}{2}$.

Match to the template

$$a = 1, \quad b = 2$$

- One recursive subproblem is created.
- Its size is at most half of the current interval.

Base case

When the interval is small enough (or empty), stop recursion and return the answer.

Small Example 2: Coins Problem

Setup

Given n coins, split them into three groups so that each group has size

$$\leq \frac{n}{3}.$$

When is the problem not small?

If $n > 1$, we treat it as a non-small problem.

Divide-and-conquer parameters

$$a = 1, \quad b = 3.$$

Reduction step

Reduce to **one** subproblem of size

$$\leq \frac{n}{3}.$$

Big Example: Multiplication

Goal

Compute $12 \cdot 34$ by splitting each number into tens and ones.

$$12 = 1 \cdot 10 + 2, \quad 34 = 3 \cdot 10 + 4$$

$$12 \cdot 34 = (1 \cdot 10 + 2)(3 \cdot 10 + 4)$$

Big Example: Multiplication

$$(1 \cdot 10 + 2)(3 \cdot 10 + 4) = (1 \cdot 3) \cdot 100 + (1 \cdot 4 + 2 \cdot 3) \cdot 10 + (2 \cdot 4)$$

Compute each part

$$(1 \cdot 3) \cdot 100 = 300, \quad (1 \cdot 4 + 2 \cdot 3) \cdot 10 = (4 + 6) \cdot 10 = 100, \quad 2 \cdot 4 = 8$$

$$12 \cdot 34 = 300 + 100 + 8 = 408$$

Takeaway

Split into high/low digits, multiply smaller parts, then combine by powers of 10.

Big Example: Multiplication of Polynomials

Setup

Treat each number as a degree-1 polynomial in x :

$$(1x^1 + 2x^0)(3x^1 + 4x^0).$$

Distribute term by term

$$(1x^1)(3x^1) + (1x^1)(4x^0) + (2x^0)(3x^1) + (2x^0)(4x^0).$$

Goal

Group terms with the same power of x .

Big Example: Multiplication of Polynomials

After grouping equal powers

$$(1x^1 + 2x^0)(3x^1 + 4x^0) = (1 \cdot 3)x^2 + (1 \cdot 4 + 2 \cdot 3)x^1 + (2 \cdot 4)x^0.$$

Coefficient view

- x^2 : $1 \cdot 3$
- x^1 : $1 \cdot 4 + 2 \cdot 3$
- x^0 : $2 \cdot 4$

Takeaway

Polynomial multiplication = multiply all pairs, then sum coefficients of equal powers.

Big Example: Multiplication of Polynomials

Input and output

Given coefficient arrays

$$a[0..n-1], \quad b[0..n-1],$$

compute

$$c[0..2n-2].$$

Definition of each output coefficient

For every $k \in \{0, \dots, 2n-2\}$,

$$c[k] = \sum_{\substack{i,j \\ i+j=k}} a[i] \cdot b[j].$$

Interpretation

To get $c[k]$, add all pairwise products whose indices sum to k .

Big Example: Multiplication of Polynomials

First coefficients

$$c[0] = a[0] \cdot b[0]$$

Big Example: Multiplication of Polynomials

First coefficients

$$c[0] = a[0] \cdot b[0]$$

$$c[1] = a[0] \cdot b[1] + a[1] \cdot b[0]$$

Big Example: Multiplication of Polynomials

First coefficients

$$c[0] = a[0] \cdot b[0]$$

$$c[1] = a[0] \cdot b[1] + a[1] \cdot b[0]$$

$$c[2] = a[0] \cdot b[2] + a[1] \cdot b[1] + a[2] \cdot b[0]$$

Big Example: Multiplication of Polynomials

First coefficients

$$c[0] = a[0] \cdot b[0]$$

$$c[1] = a[0] \cdot b[1] + a[1] \cdot b[0]$$

$$c[2] = a[0] \cdot b[2] + a[1] \cdot b[1] + a[2] \cdot b[0]$$

Pattern

- Index k defines a diagonal.
- Collect all (i, j) with $i + j = k$.
- Sum their products.

Takeaway

Polynomial multiplication is coefficient convolution.

Warm-Up: Polynomials as Algebra

Addition

$$\begin{aligned}(2x^2 + x) + (3x + 5) \\ = 2x^2 + (x + 3x) + 5 = 2x^2 + 4x + 5\end{aligned}$$

Reminder

$$1 = x^0, \quad 5 = 5x^0$$

Multiplication (Distribute)

$$\begin{aligned}(2x^2 + x)(3x + 5) \\ = 2x^2 \cdot 3x + 2x^2 \cdot 5 + x \cdot 3x + x \cdot 5 \\ = 6x^3 + 10x^2 + 3x^2 + 5x = 6x^3 + 13x^2 + 5x\end{aligned}$$

Coefficient View (for Divide-and-Conquer)

Addition = Component-wise Sum

Use coefficient order $(x^0, x^1, x^2, \dots)$:

$$2x^2 + x \longleftrightarrow [0, 1, 2]$$

$$3x + 5 \longleftrightarrow [5, 3, 0]$$

$$[0, 1, 2] + [5, 3, 0] = [5, 4, 2]$$

$$\Longleftrightarrow 2x^2 + 4x + 5$$

Multiplication = Convolution

$$C(x) = A(x) \cdot B(x)$$

Example:

$$A = [1, 1, 1], \quad B = [1, 2, 2]$$

$$C = [1, 3, 5, 4, 2, 0]$$

(Last zero is just padding.)

Key idea

Polynomial multiplication is not component-wise product; it mixes terms by degree.

Trivial Algorithm: Multiplication of Polynomials

Problem

Given coefficient arrays $a[0..n-1]$ and $b[0..n-1]$, compute $c[0..2n-2]$ such that

$$c[k] = \sum_{\substack{i,j \\ i+j=k}} a[i] \cdot b[j].$$

Direct algorithm

1. Initialize $c[0..2n-2] \leftarrow 0$
2. For $i \leftarrow 0$ to $n-1$ do
3. For $j \leftarrow 0$ to $n-1$ do
4. $c[i+j] \leftarrow c[i+j] + a[i] \cdot b[j]$
5. End For
6. End For

Interpretation

Multiply every pair of coefficients and accumulate by degree index $i+j$.

Runtime of the Trivial Algorithm

Operation count

- Outer loop runs n times.
- Inner loop runs n times per outer iteration.
- Each update

$$c[i + j] \leftarrow c[i + j] + a[i] \cdot b[j]$$

takes $\Theta(1)$.

Total time

$$n \cdot n \cdot \Theta(1) = \Theta(n^2).$$

Next question

Can we multiply polynomials faster than $\Theta(n^2)$?

Karatsuba Algorithm: Base Case (size = 1)

When do we stop recursing?

If each input has one digit (or one coefficient), do a direct multiplication.

$$a = [u], \quad b = [v] \implies c = [u \cdot v]$$

Example

$$a = [2], \quad b = [3] \implies c = [6]$$

Base-case rule

return $a[0] \cdot b[0]$

Karatsuba Algorithm: Recursive Case (size > 1)

Divide step

When the inputs are not in the base case, split each input into two halves:

$$a = a_H \cdot B^m + a_L, \quad b = b_H \cdot B^m + b_L$$

Here, a_H, b_H are the high halves, a_L, b_L are the low halves, and m is about half of the current size.

Input a

$$a \longrightarrow (a_H, a_L)$$

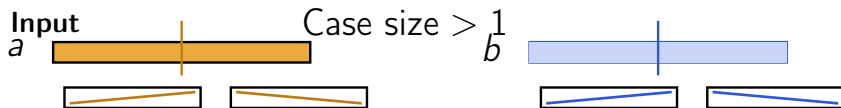
Input b

$$b \longrightarrow (b_H, b_L)$$

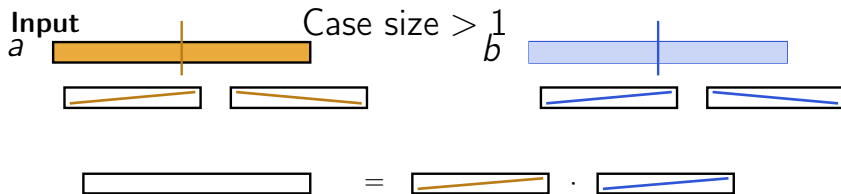
Decision rule

If size > 1, apply this split and continue recursively on smaller parts.

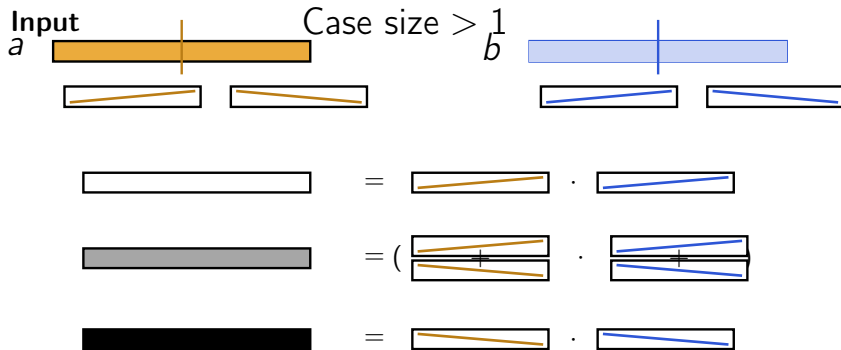
Karatsuba Algorithm: Recursive Case ($\text{size} > 1$)



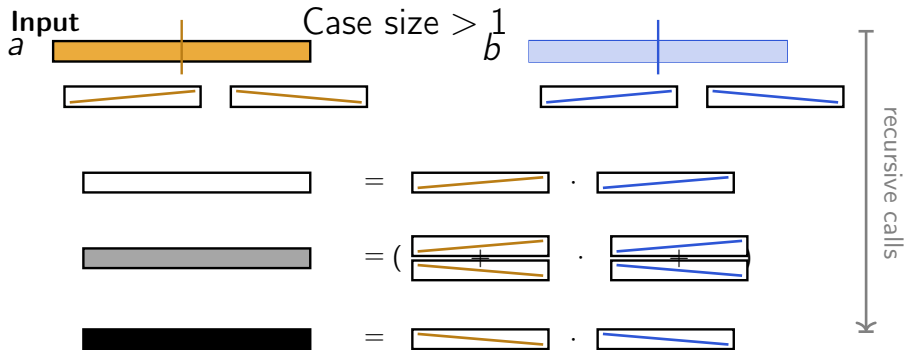
Karatsuba Algorithm: Recursive Case ($\text{size} > 1$)



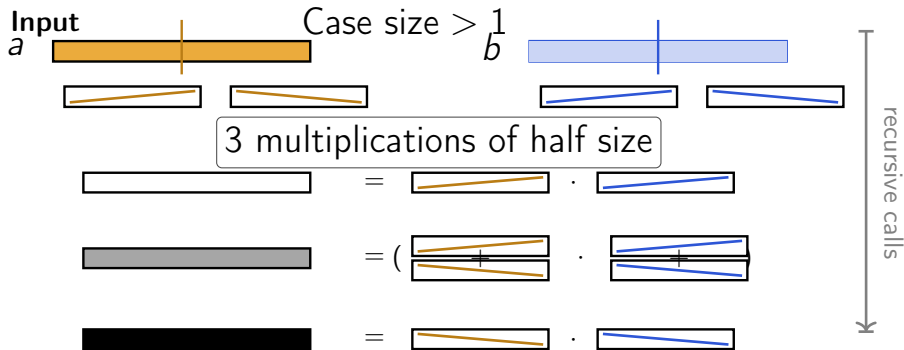
Karatsuba Algorithm: Recursive Case ($\text{size} > 1$)



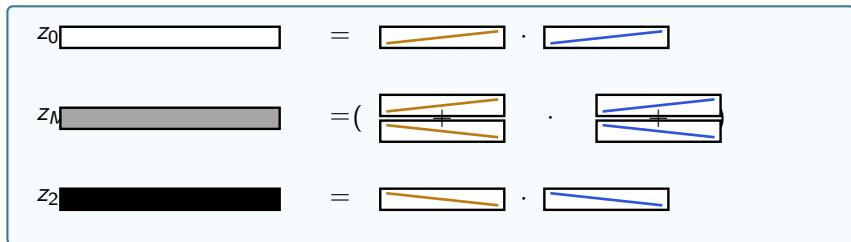
Karatsuba Algorithm: Recursive Case ($\text{size} > 1$)



Karatsuba Algorithm: Recursive Case ($\text{size} > 1$)



Karatsuba Algorithm: Combine the Three Products



Karatsuba Algorithm: Recursive Case ($\text{size} > 1$)

After splitting into halves

$$a = a_H \cdot B^m + a_L, \quad b = b_H \cdot B^m + b_L$$

compute only three products of half size:

$$z_2 = a_H b_H, \quad z_0 = a_L b_L, \quad z_M = (a_H + a_L)(b_H + b_L).$$

Why this is the key idea

Each of z_2, z_0, z_M is a recursive multiplication on smaller inputs (about half the size).

Recursive work per level

3 multiplications of half size

Karatsuba Algorithm: Combine the Three Products

Recover the middle term without a fourth multiplication

$$(a_H + a_L)(b_H + b_L) = a_H b_H + a_H b_L + a_L b_H + a_L b_L$$

so

$$a_H b_L + a_L b_H = z_M - z_2 - z_0.$$

Final recombination

$$ab = z_2 \cdot B^{2m} + (z_M - z_2 - z_0) \cdot B^m + z_0.$$

Decision rule

If size > 1: split, compute z_2, z_0, z_M recursively, then combine with the formula above.

Karatsuba Algorithm: Recursive Procedure

Input

Two length- n vectors (digits or coefficients):

$$a[0..n-1], \quad b[0..n-1].$$

Base case

If $n = 1$, return

$$a[0] \cdot b[0].$$

Recursive case ($n > 1$)

Let $s = \frac{n}{2}$, then split:

$$a_0 = a[0..s-1], \quad a_1 = a[s..n-1],$$

$$b_0 = b[0..s-1], \quad b_1 = b[s..n-1].$$

Three recursive multiplications

$$c_0 = \text{multiply}(a_0, b_0), \quad c_1 = \text{multiply}(a_0 + a_1, b_0 + b_1), \quad c_2 = \text{multiply}(a_1, b_1).$$

Karatsuba Algorithm: Recursive Procedure

Algorithm sketch

multiply($a[0..n-1]$, $b[0..n-1]$)

- If $n = 1$, return $a[0] \cdot b[0]$.
- Otherwise, let $s = \frac{n}{2}$ and split:

$$a_0 = a[0..s-1], \quad a_1 = a[s..n-1], \quad b_0 = b[0..s-1], \quad b_1 = b[s..n-1].$$

- Compute three recursive products:

$$c_0 = \text{multiply}(a_0, b_0), \quad c_1 = \text{multiply}(a_0 + a_1, b_0 + b_1), \quad c_2 = \text{multiply}(a_1, b_1).$$

- Return

$$c_0 + \text{shift}(c_1 - c_0 - c_2, s) + \text{shift}(c_2, n).$$

Karatsuba Algorithm: Running Time

Set up the recurrence

Let $T(n)$ be the running time for inputs of size n .

$$T(1) = \Theta(1)$$

For $n > 1$:

$$T(n) = 3 T\left(\frac{n}{2}\right) + \Theta(n).$$

Why this form?

- $3 T(n/2)$: three recursive multiplications on half-size inputs.
- $\Theta(n)$: additions, subtractions, splitting, and shifts.

Assumption

Assume $n = 2^d$, so each split is even.

Next step

How do we solve

$$T(n) = 3 T\left(\frac{n}{2}\right) + \Theta(n)?$$

Karatsuba Algorithm: Running Time (Expansion)

Recurrence

$$T(1) = \Theta(1), \quad T(n) = 3 T\left(\frac{n}{2}\right) + \Theta(n) \quad (n > 1).$$

Unroll a few steps

$$\begin{aligned} T(n) &= 3\left(3T\left(\frac{n}{4}\right) + \Theta\left(\frac{n}{2}\right)\right) + \Theta(n) \\ &= 3^2 T\left(\frac{n}{2^2}\right) + 3\Theta\left(\frac{n}{2}\right) + \Theta(n) \\ &\vdots \\ &= 3^d T\left(\frac{n}{2^d}\right) + \sum_{i=0}^{d-1} 3^i \Theta\left(\frac{n}{2^i}\right). \end{aligned}$$

Karatsuba Algorithm: Final Asymptotic Bound

Substitute $n = 2^d$

Because $\frac{n}{2^d} = 1$, we get

$$T(n) = 3^d \Theta(1) + \Theta\left(n \sum_{i=0}^{d-1} \left(\frac{3}{2}\right)^i\right).$$

The geometric sum is dominated by its largest term, so

$$T(n) = \Theta(3^d).$$

Rewrite in terms of n

$$3^d = 3^{\log_2 n} = n^{\log_2 3}.$$

Therefore,

$$T(n) = \Theta\left(n^{\log_2 3}\right).$$

Polynomial Multiplication: Comparing Methods

Asymptotic running times

- Trivial method: $\Theta(n^2)$
- Karatsuba: $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$
- FFT-based methods: $O(n \log n)$

Asymptotic order (for large n)

$$n \log n \ll n^{1.585} \ll n^2.$$

At $n = 1024$

$$n^2 = 1,048,576 (> 10^6)$$

$$n^{1.585} \approx 5.9 \times 10^4 (< 60,000)$$

Caveat

Big- O hides constants, so asymptotically faster methods are not always best for very small inputs.

Practical Rule: Which Algorithm to Run?

Decision rule in practice

- If n is small, run the trivial $\Theta(n^2)$ algorithm.
- Otherwise, switch to Karatsuba.

Why this hybrid works

- Trivial method has low overhead and simple operations.
- Karatsuba wins after crossing a size threshold.
- The threshold depends on implementation and hardware.

Takeaway

small $n \Rightarrow$ trivial

larger $n \Rightarrow$ Karatsuba

FFT methods become preferable only for sufficiently large n .

Master Theorem

Standard divide-and-conquer recurrence

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

- a : number of recursive subproblems,
- $\frac{n}{b}$: size of each subproblem,
- $f(n)$: non-recursive work (divide/combine).

Master Theorem: Important Caveats

Rounding Variant

The theorem also applies when $\frac{n}{b}$ is replaced by

$$\left\lfloor \frac{n}{b} \right\rfloor \quad \text{or} \quad \left\lceil \frac{n}{b} \right\rceil .$$

Constants Condition

a and b are constants (typically $a \geq 1$, $b > 1$).

Master Theorem: The Three Cases

$$T(n) = a T\left(\frac{n}{b}\right) + f(n), \quad c = \log_b a$$

Case A (leaf-dominated)

If $f(n) = O(n^{c-\varepsilon})$ for some $\varepsilon > 0$, then

$$T(n) = \Theta(n^c).$$

Case B (balanced)

If $f(n) = \Theta(n^c)$, then

$$T(n) = \Theta(n^c \log n).$$

Case C (root-dominated)

If $f(n) = \Omega(n^{c+\varepsilon})$ for some $\varepsilon > 0$, and (regularity condition)

$$a f\left(\frac{n}{b}\right) \leq k f(n) \quad \text{for some } k < 1,$$

then

$$T(n) = \Theta(f(n)).$$

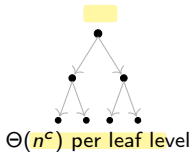
Master Theorem: Visual Intuition

Recursion tree viewpoint

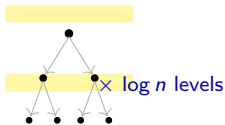
$$T(n) = a T\left(\frac{n}{b}\right) + f(n), \quad c = \log_b a$$

At level i , there are a^i subproblems of size n/b^i , so the total work on that level is

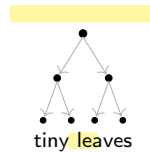
$$a^i f\left(\frac{n}{b^i}\right).$$



Case A
 $f(n)$ is smaller
than leaf work



Case B
every level contributes
about the same



Case C
the root dominates
all lower levels

Example: $T(n) = 3T(n/2) + \Theta(n)$

Match parameters

$$a = 3, \quad b = 2, \quad f(n) = \Theta(n)$$

$$c = \log_b a = \log_2 3 \approx 1.58$$

Compare $f(n)$ with n^c

$$f(n) = \Theta(n) = \Theta(n^1)$$

So we can take $c' = 1$, and

$$1 < \log_2 3.$$

Apply Case 1

Since $f(n) = O(n^{c'})$ for $c' < c$, Case 1 applies.

$$T(n) = \Theta\left(n^{\log_2 3}\right).$$

Master Theorem Example: Binary Search

Recurrence

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1)$$

Match parameters

$$a = 1, \quad b = 2, \quad f(n) = \Theta(1)$$

$$c = \log_b a = \log_2 1 = 0$$

Compare $f(n)$ with n^c

$$n^c = n^0 = 1$$

$$f(n) = \Theta(1) = \Theta(n^c)$$

So this is **Master Theorem Case 2**.

Conclusion

$$T(n) = \Theta(n^c \log n) = \Theta(n^0 \log n) = \Theta(\log n)$$

Merge Sort via Master Theorem

Recurrence

$$T(n) = 2T(n/2) + \Theta(n)$$

- $a = 2$
- $b = 2$
- $f(n) = \Theta(n)$
- $c = \log_b a = \log_2 2 = 1$
- Case A does not apply ($f(n)$ is not $O(n^{c'})$ for $c' < 1$).
- Case B applies since $f(n) = \Theta(n^c) = \Theta(n)$.

Conclusion

$$T(n) = \Theta(n^c \log n) = \Theta(n \log n).$$

Karatsuba Recurrence: already in Master Theorem form

Recurrence

$$T(n) = 3T(n/2) + \Theta(n)$$

Match to $T(n) = aT(n/b) + f(n)$

$$a = 3, \quad b = 2, \quad f(n) = \Theta(n)$$

$$c = \log_b a = \log_2 3 \approx 1.585$$

Key comparison

$$f(n) = \Theta(n) = O(n^{c'}) \text{ for } c' = 1 < \log_2 3$$

So this is **Master Theorem Case A** (f is polynomially smaller than n^c).

Karatsuba memory complexity

Recurrence for extra memory

$$M(n) = M(n/2) + \Theta(n)$$

Match the Master Theorem

$$a = 1, \quad b = 2, \quad f(n) = \Theta(n)$$

$$c = \log_b a = \log_2 1 = 0$$

Check all cases

- Case A: no, because $f(n) = \Theta(n)$ is not $O(n^{c'})$ for any $c' < 0$.
- Case B: no, because $f(n) \neq \Theta(n^c) = \Theta(1)$.
- Case C: yes, since $f(n) = \Theta(n) = \Omega(n^{c'})$ for $c' = 1 > 0 = c$.

Conclusion

$$M(n) = \Theta(f(n)) = \Theta(n).$$